

Computer Vision & AI

SW 04: Image Compression

8. März 2024

Agenda

Compression

- Lossless Compression
 - Run Length Encoding (RLE)
 - Lempel-Ziv Compression
 - Lempel-Ziv 77 (LZ77)
 - Lempel-Ziv 78 (LZ78)
 - Lempel-Ziv-Welch (LZW)
 - Huffman Coding
- Lossy Compression
 - Discrete Cosine Transform (DCT)

Goals

- Knowledge of some Compression Methods
- Application to simple Problems
- Knowledge of
 - Advantages
 - Disadvantages
 - Applications

Applications

Advantages

- reduction of data volume
- less storage necessary
- less transmission time

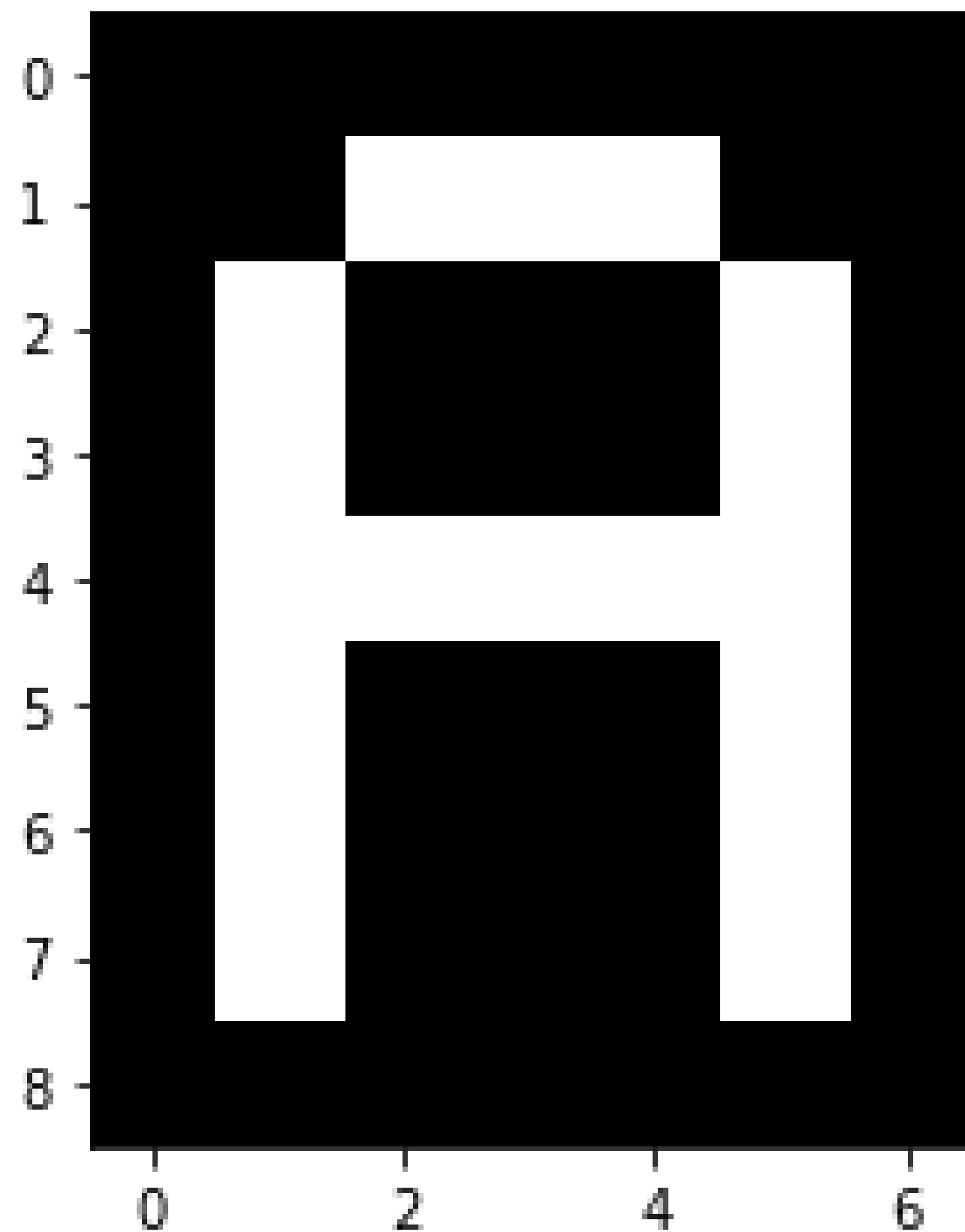
Fields of Application

- Images
 - Digital Photographs (JPEG)
 - Videos (MPEG)
- Audio (MP3)
- Data
 - Storage
 - Communication
 - Transmission
 - Reception

Software

- RLE:
 - BMP (Windows Bitmap)
 - FAX
- LZ77
GZIP, ZIP
- LZ78
GIF
- LZW
compress
- Huffman Coding
pack
-
- DCT
 - JPEG
 - MPEG

Run Length Encoding



- Binary image
- 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 0, 0,
0, 1, 0, 0, 0, 1, 0,
0, 1, 0, 0, 0, 1, 0,
0, 1, 1, 1, 1, 1, 0,
0, 1, 0, 0, 0, 1, 0,
0, 1, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0
- 9 x 7 = 63 pixels
- sequences of identical pixels
- sequences lengths are
9, 3, 3, 1, 3, 1, 2, 1, 3, 1, 2,
5, 2, 1, 3, 1, 2, 1, 3, 1, 2, 1,
3, 1, 8
- store 25 lengths
instead of 63 pixel values
- the longer the sequences
the better the compression
- worst case: checkerboard

Python notebook: `run-length-encoding-letter-a.ipynb`

https://en.wikipedia.org/wiki/Run-length_encoding

<https://de.wikipedia.org/wiki/Lauf%C3%A4ngenkodierung>

Dictionary

Dictionary contains

- some or all letters of the alphabet
- as many digrams as possible

Example

- String "abracadabra"
- $11 \times 8 \text{ Bit} = 88 \text{ Bit}$
- Dictionary:
 - Alphabet:
 - 0 (000): a
 - 1 (001): b
 - 2 (010): c
 - 3 (011): d
 - 4 (100): r
 - Digrams:
 - 5 (101): ab
 - 6 (110): ac
 - 7 (111): ad

Encoding

- Decomposition of string in letters and digrams:
ab r ac ad ab r a
- using indices instead of letters and digrams:
101 100 110 111 101 100 000
- $7 \times 3 \text{ Bit} = 21 \text{ Bit}$

https://en.wikipedia.org/wiki/Dictionary_coder

<https://de.wikipedia.org/wiki/W%C3%B6rterbuchkompression>

LZ77: Compress (Encode)

- idea: replace substrings already occurred in text by codes
- text to be processed next in look-ahead buffer (size 4)
- text already processed in search buffer (size 6)
- look for coincident substrings in lookahead and search buffer
- code: (m, n, c)
 - index (m) and length (n) of substring in search buffer, m = n = 0: not found
 - last output character

Example: Compress "abracabrabra" (abra c a b ra bra)

search buffer						look-ahead buffer				code	
6	5	4	3	2	1						
						a	b	r	a	(0, 0, a)	
					a	b	r	a	c	(0, 0, b)	
				a	b	r	a	c	a	(0, 0, r)	
			a	b	r	a	c	a	b	(3, 1, c)	
	a	b	r	a	c	a	b	r	a	(5, 3, a)	
a	c	a	b	r	a	b	r	a		(3, 3, -)	

LZ77: Decompress (Decode)

substrings in buffer: "..."
character in code: '.'

- 1. no string, 'a'
- 2. no string, 'b'
- 3. no string, 'r'
- 4. "a" at index 3, 'c'
- 5. "abr" at index 5, 'a'
- 6. "bra" at index 3, no character

input		buffer						output			
#	code	6	5	4	3	2	1				
1	(0, 0, a)							a			
2	(0, 0, b)						a	b			
3	(0, 0, r)					a	b	r			
4	(3, 1, c)				a	b	r	a	c		
5	(5, 3, a)		a	b	r	a	c	a	b	r	a
6	(3, 3, -)	a	c	a	b	r	a	b	r	a	

<https://en.wikipedia.org/wiki/LZ77> and [LZ78#LZ77](https://en.wikipedia.org/wiki/LZ78#LZ77)
<https://de.wikipedia.org/wiki/LZ77>

LZ78: Compress (Encode)

Example: Compress "abbcbcababcaabcaab" (a b bc bca ba bcaa bcaab)

Code: (m, c)

- m: index of string in dictionary, 0 = no string
- last output character

1. a not in dictionary, add a
2. b not in dictionary, add b
3. b in dictionary (index 2), bc not, add bc
4. bc in dictionary (index 3), bca not, add bca
5. b in dictionary (index 2), a not, add ba
6. bca in dictionary (index 4), bcaa not, add bcaa
7. bcaa in dictionary (index 6), bcaab not, add bcaab

dictionary	
index	string
1	a
2	b
3	bc
4	bca
5	ba
6	bcaa
7	bcaab

code	
#	output
1	(0, a)
2	(0, b)
3	(2, c)
4	(3, a)
5	(2, a)
6	(4, a)
7	(6, b)

LZ78: Decompress (Decode)

substrings from dictionary: "..."
character from code: '.'

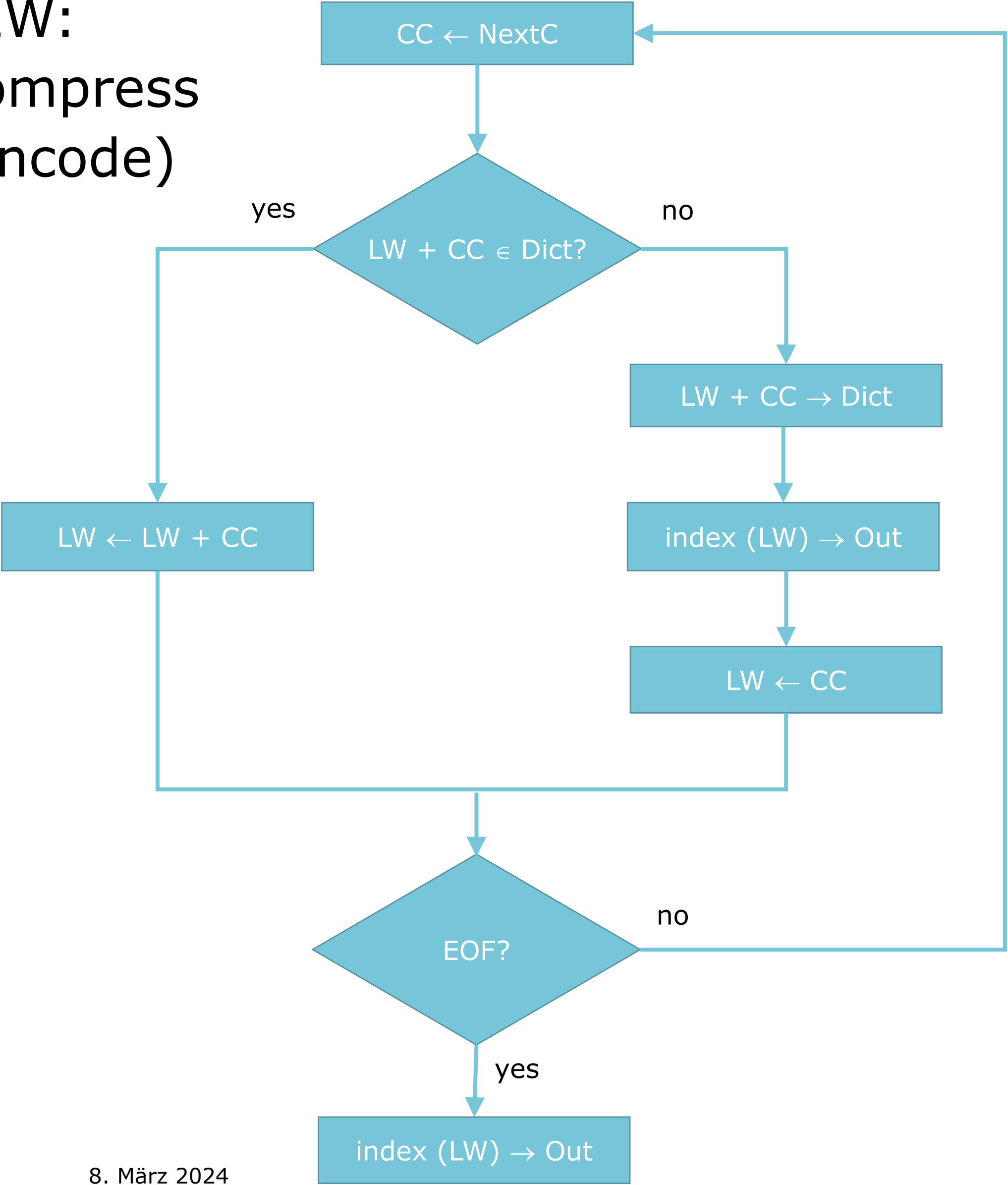
every output is added
to the dictionary.

- 1. no string, 'a'
- 2. no string, 'b'
- 3. "b" at index 2, 'c'
- 4. "bc" at index 3, 'a'
- 5. "b" at index 2, 'a'
- 6. "bca" at index 4, 'a'
- 7. "bcaa" at index 6, 'b'

input		dictionary		output
#	code	index	string	
1	(0, a)	1	a	a
2	(0, b)	2	b	b
3	(2, c)	3	bc	bc
4	(3, a)	4	bca	bca
5	(2, a)	5	ba	ba
6	(4, a)	6	bcaa	bcaa
7	(6, b)	7	bcaab	bcaab

[https://en.wikipedia.org/wiki/LZ77 and LZ78#LZ78](https://en.wikipedia.org/wiki/LZ77_and_LZ78#LZ78)
<https://de.wikipedia.org/wiki/LZ78>

LZW:
Compress
(Encode)



LW	CC	LW+CC	Dict	Out
- / r	r	r		
<u>r</u> / a	a	ra	256: ra	114
<u>a</u> / b	b	ab	257: ab	97
<u>b</u> / a	a	ba	258: ba	98
<u>a</u> / r	r	ar	259: ar	97
<u>r</u> / b	b	rb	260: rb	114
b / ba	a	ba		
<u>ba</u> / r	r	bar	261: bar	258
r / rb	b	rb		
<u>rb</u> / a	a	rba	262: rba	260
a / ar	r	ar		
<u>ar</u> / b	b	arb	263: arb	259
b / ba	a	ba		
ba / bar	r	bar		
<u>bar</u> / a	a	bara	264: bara	261
<u>a</u>	EOF	-	-	97

LZW: Decompress

```
code ← NextCode
w ← Dict [code]
w → Out

while (code ← NextCode) {
  if (code ∈ Dict) {
    tmp ← w
    w ← Dict (code)
    tmp + w [0] → Dict
  } else {
    w + w [0] → Dict
    w ← Dict (code)
  }
  w → Out
}
```

code	tmp	w	Dict	Out
114		r		r
97	r	a	256: ra	a
98	a	b	257: ab	b
97	b	a	258: ba	a
114	a	r	259: ar	r
258	r	ba	260: rb	ba
260	ba	rb	261: bar	rb
259	rb	ar	262: rba	ar
261	ar	bar	263: arb	bar
97	bar	a	264: bara	a

<https://en.wikipedia.org/wiki/Lempel%E2%80%93Ziv%E2%80%93Welch>
<https://de.wikipedia.org/wiki/Lempel-Ziv-Welch-Algorithmus>

LZW: Exercise

Compress babababara

Start Dictionary:

- 1: a
- 2: b
- 3: r

Solution: 2, 1, 4, 6, 1, 3, 1

Decompress 2, 1, 4, 6, 1, 3, 1

Start Dictionary:

- 1: a
- 2: b
- 3: r

Solution: babababara

Huffman Coding

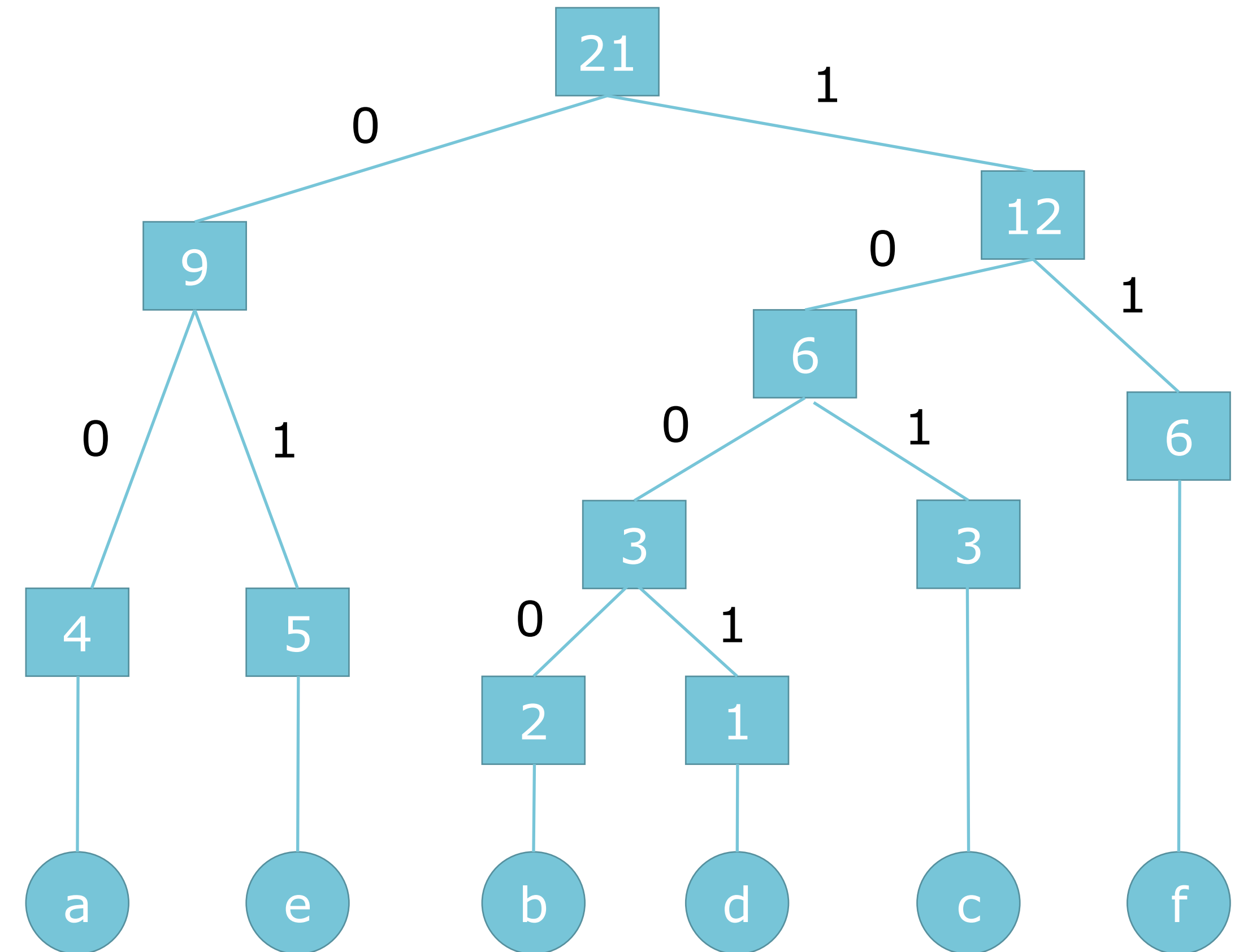
- count the frequencies of all characters in the text
- construct a forest
 - with one tree for every character
 - every tree has
 - one single node: the character itself
 - the frequency of the character as attribute
- reduce the forest to one tree
 - look for the two trees with the smallest frequencies
 - combine them as left and right subtree of a new tree
 - the frequency of the new tree is the sum both subtrees' frequencies
 - repeat the reduction until only tree is left
- the result of the reduction process is the Huffman tree
- for every node of the Huffman tree denote
 - the edge to the left subtree with 0 and
 - the edge to the right subtree with 1
- for each character
 - follow the way
 - from the Huffman tree's root
 - to the character's leaf
 - collect the zeros and ones at the edges
 - to get the code of the character

https://en.wikipedia.org/wiki/Huffman_coding

<https://de.wikipedia.org/wiki/Huffman-Kodierung>

Huffman Coding

- Example: aeebafabacecffcdefeff, 21 Byte = 168 Bit
- Frequencies:
a: 4, b: 2, c: 3, d: 1, e: 5, f: 6
- Huffman tree (see right)
- Codes:
a: 00, b: 1000, c: 101, d: 1001, e: 01, f: 11
- Compressed:
00 01 01 1000 00 11 00 1000 00 101
01 101 11 11 101 1001 01 11 01 11
11 (51 Bit)
- Decompressed:
a e e b a f a b a c
e c f f c d e f e f f
f



Discrete Cosine Transform

Introduction

- Sound
 - vs noise
 - as a mixture of sine waves (synthesis)
- Analysis:
 - identify all sine waves of a sound
 - (Fourier) analysis

https://en.wikipedia.org/wiki/Fourier_analysis

<https://de.wikipedia.org/wiki/Fourier-Analysis>

Python Notebook: `fourier.ipynb`

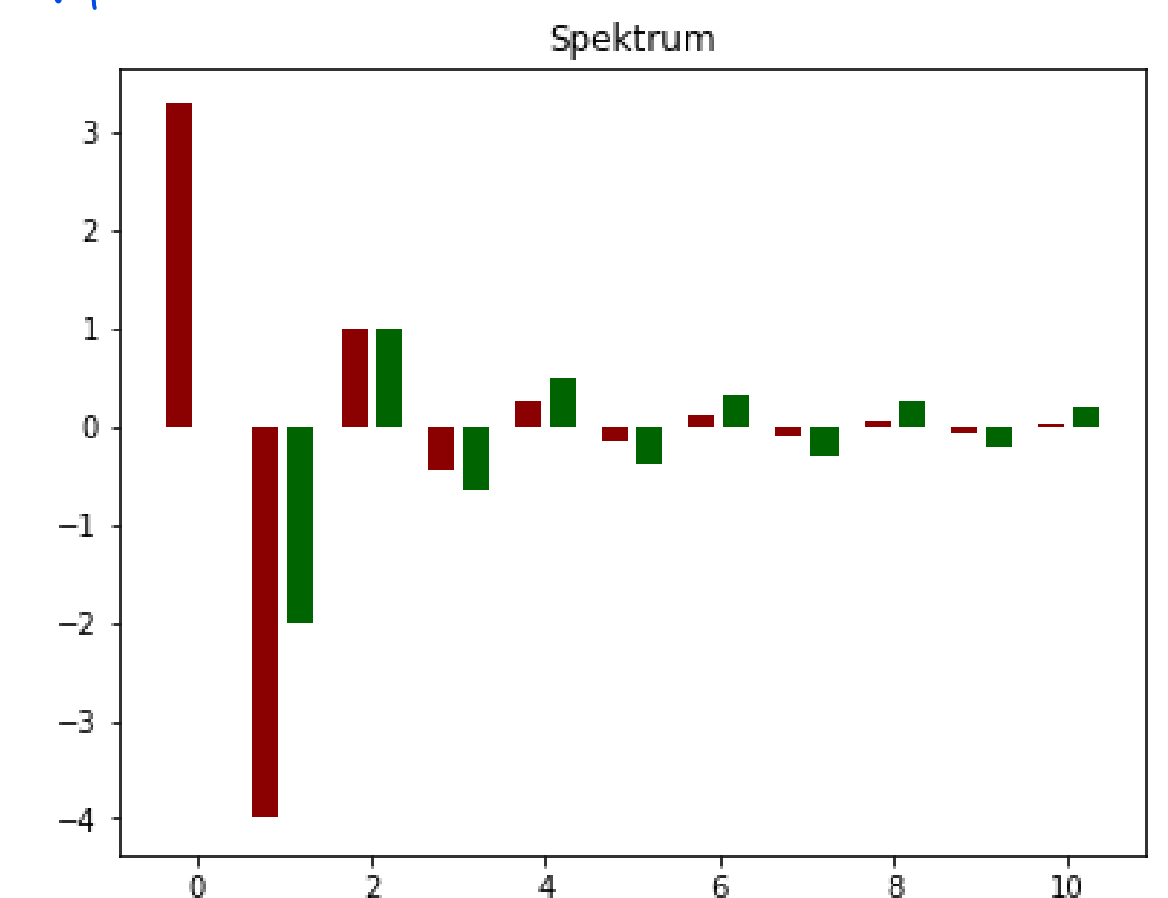
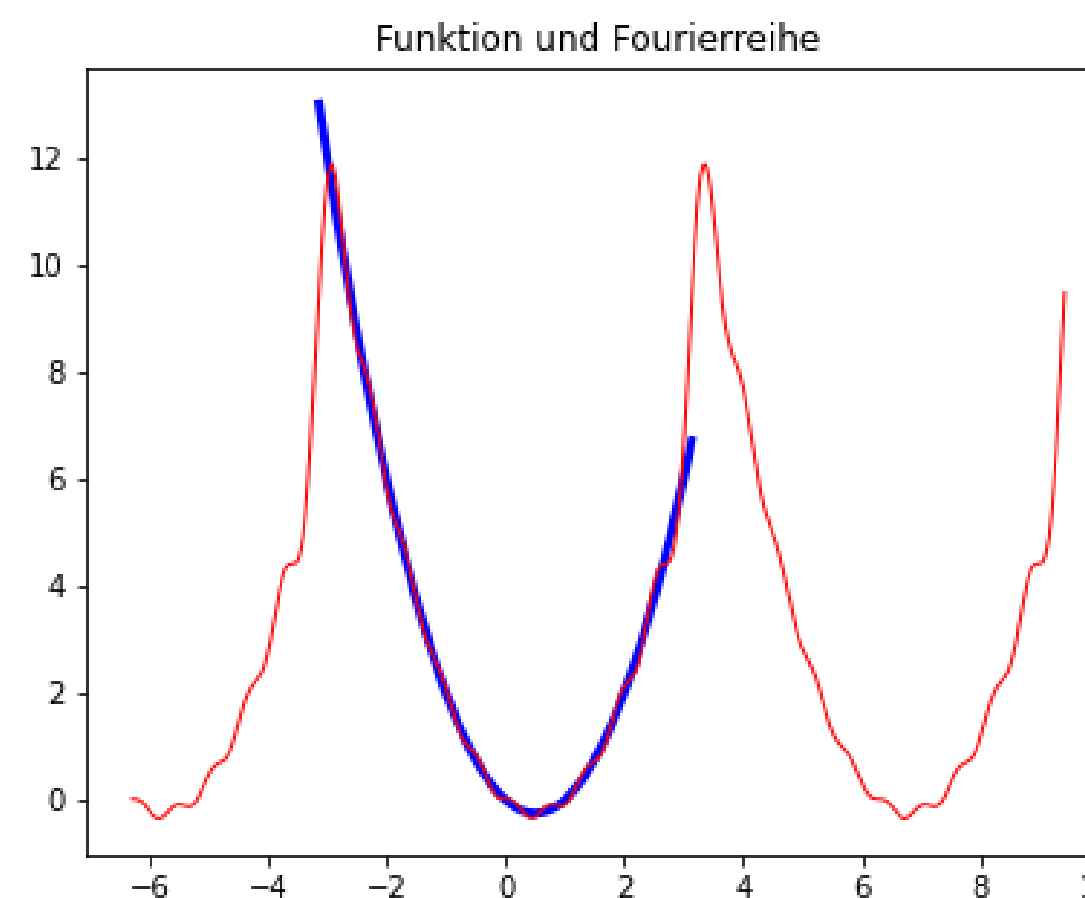
Some Mathematics

$$f(x) \approx \frac{a_0}{2} + \sum_{k=1}^n a_k \cos(kx) + b_k \sin(kx) \quad \text{sound}$$

$$a_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(kx) dx \quad \text{cosine amplitudes}$$

$$b_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(kx) dx \quad \text{sine amplitudes}$$

Sin



Discrete Cosine Transform

- start with Fourier analysis
- switch to Fourier transform (complex numbers necessary)
- with DCT complex numbers can be avoided
- use interval $[0, \pi]$ instead of $[-\pi, \pi]$
- omit sine, restrict to cosine
- replace integrals with sums
- use discretization of function
- see slides from Prof. Dr. J. Bürgler
- python notebook: `dct.ipynb`

